

EXPERIMENT 10

*Write a Python Program to Implement the A Search Algorithm**

Aim

To develop a Python program to implement the **A* (A-Star) Search Algorithm** for finding the shortest path between a start node and a goal node in a graph.

Objective

- To understand the concept of the **A* Search Algorithm**.
- To implement A* Search using Python.
- To find the optimal path between the source and destination.
- To understand the role of heuristic functions in Artificial Intelligence.

Theory

The **A* (A-Star) Search Algorithm** is one of the most efficient **informed search algorithms** used in Artificial Intelligence. It combines the advantages of **Uniform Cost Search** and **Greedy Best-First Search** by considering both the actual cost from the start node and the estimated cost to the goal node.

The evaluation function used in A* is:

$$f(n) = g(n) + h(n)$$

Where:

- **g(n)** = Actual cost from the start node to the current node.
- **h(n)** = Heuristic estimate from the current node to the goal node.
- **f(n)** = Total estimated cost of the path.

The algorithm always expands the node with the smallest **f(n)** value, ensuring that the shortest path is found when the heuristic is admissible.

Algorithm

Step 1: Define the graph with nodes and edge costs.

Step 2: Define the heuristic value for each node.

Step 3: Insert the start node into the open list.

Step 4: Select the node having the minimum **f(n)** value.

Step 5: If the selected node is the goal node, display the shortest path and stop.

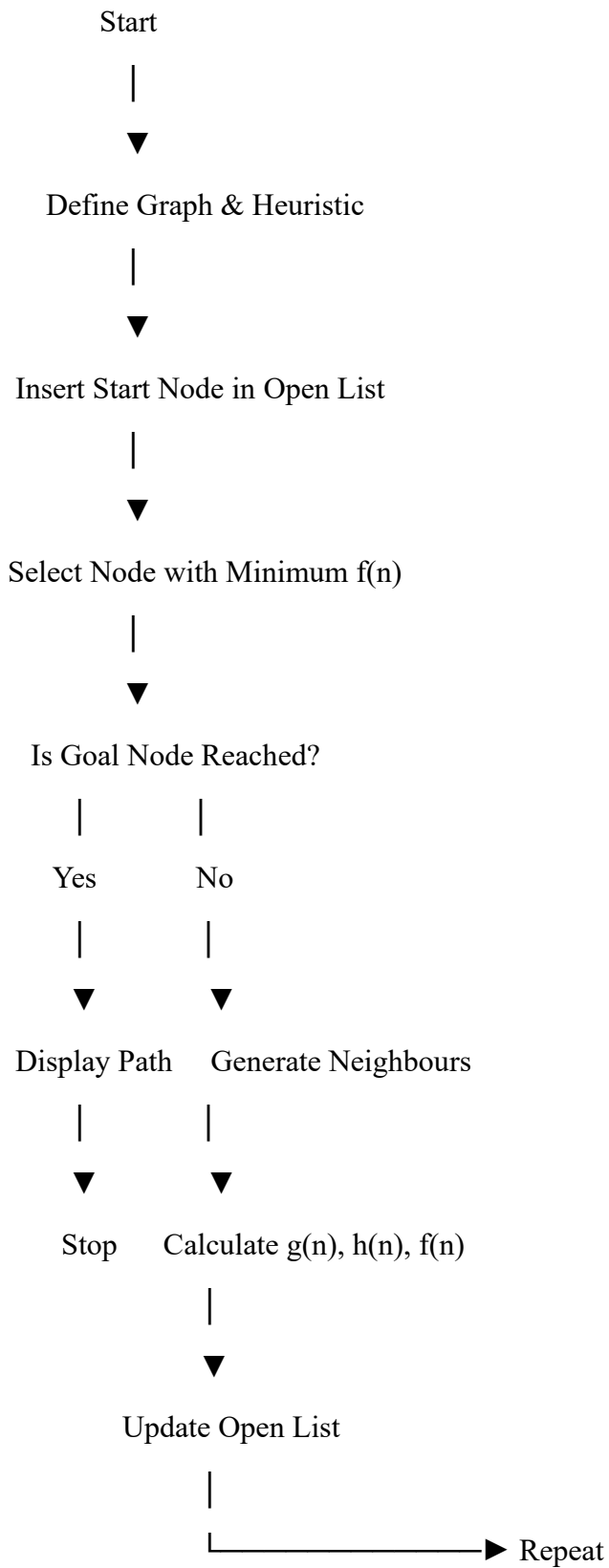
Step 6: Otherwise, generate all neighboring nodes.

Step 7: Calculate the **g(n)**, **h(n)**, and **f(n)** values for each neighboring node.

Step 8: Update the open list with unexplored nodes.

Step 9: Repeat Steps 4 to 8 until the goal node is reached.

Flowchart



Python Program

```
graph = {  
    'A': [('B', 1), ('C', 3)],  
    'B': [('D', 3), ('E', 6)],  
    'C': [('F', 5)],  
    'D': [('G', 2)],  
    'E': [('G', 1)],  
    'F': [('G', 2)],  
    'G': []  
}
```

```
heuristic = {  
    'A': 7,  
    'B': 6,  
    'C': 4,  
    'D': 2,  
    'E': 1,  
    'F': 2,  
    'G': 0  
}
```

```
def astar(start, goal):
```

```
    open_list = [(start, 0, [start])]
```

```
    visited = set()
```

```
    while open_list:
```

```
        open_list.sort(key=lambda x: x[1] + heuristic[x[0]])
```

```
node, cost, path = open_list.pop(0)
```

```
if node == goal:
```

```
    return path, cost
```

```
if node in visited:
```

```
    continue
```

```
visited.add(node)
```

```
for neighbour, weight in graph[node]:
```

```
    if neighbour not in visited:
```

```
        open_list.append(
            (neighbour,
             cost + weight,
             path + [neighbour])
        )
```

```
return None
```

```
path, cost = astar('A', 'G')
```

```
print("Shortest Path :", " -> ".join(path))
```

```
print("Total Cost :", cost)
```

Sample Output

Shortest Path : A -> B -> D -> G

Total Cost : 6

Explanation

The graph contains seven nodes: **A, B, C, D, E, F, and G.**

The algorithm starts from **Node A.**

At every step, it selects the node with the smallest **f(n)** value, where:

$$\mathbf{f(n) = g(n) + h(n)}$$

Using the heuristic values, the algorithm explores the most promising path first.

The shortest path obtained is:

A → B → D → G

The total path cost is **6**, which is the minimum possible cost.

Advantages

- Finds the optimal path when the heuristic is admissible.
- Faster than uninformed search algorithms.
- Reduces unnecessary node exploration.
- Efficient for large search spaces.
- Widely used in Artificial Intelligence applications.

Disadvantages

- Performance depends on the quality of the heuristic.
- Requires additional memory to store the open and closed lists.
- Can become slow if the heuristic is poor.

Applications

- GPS Navigation Systems.
- Robot Path Planning.
- Video Game Pathfinding.
- Network Routing.
- Autonomous Vehicles.
- Artificial Intelligence Search Problems.

Result

The Python program successfully implemented the **A* Search Algorithm** and determined the shortest path from the start node to the goal node along with the minimum path cost.

Conclusion

The **A* Search Algorithm** is one of the most efficient search algorithms in Artificial Intelligence because it combines the actual path cost with a heuristic estimate to find the optimal solution. It significantly reduces the search space compared to uninformed search algorithms while guaranteeing the shortest path when an admissible heuristic is used. Due to its efficiency and accuracy, A* is widely applied in robotics, navigation systems, game development, and intelligent pathfinding applications.

```
1 graph = {
2     'A': [('B', 1), ('C', 3)],
3     'B': [('D', 3), ('E', 6)],
4     'C': [('F', 5)],
5     'D': [('G', 2)],
6     'E': [('G', 1)],
7     'F': [('G', 2)],
8     'G': []
9 }
10
11 heuristic = {
12     'A': 7,
13     'B': 6,
14     'C': 4,
15     'D': 2,
16     'E': 1,
17     'F': 2,
18     'G': 0
19 }
20
21 def astar(start, goal):
22     open_list = [(start, 0, [start])]
23     visited = set()
24
25     while open_list:
26
27         open_list.sort(key=lambda x: x[1] + heuristic[x[0]])
28
29         node, cost, path = open_list.pop(0)
30
31         if node == goal:
32             return path, cost
33
34         if node in visited:
35             continue
36
37         visited.add(node)
38
39         for neighbour, weight in graph[node]:
40
41             if neighbour not in visited:
42
43                 open_list.append(
44                     (neighbour,
45                      cost + weight,
46                      path + [neighbour])
47                 )
48
49     return None
50
51
52 path, cost = astar('A', 'G')
53
54 print("Shortest Path :", " -> ".join(path))
55 print("Total Cost :", cost)
56
57
```

```
● PS C:\Users\Varshini M\AppData\Local\Programs\Microsoft
/OneDrive/Desktop/Here!!!/ARTIFICIAL INTELLIGENCE/VSC
Shortest Path : A -> B -> D -> G
Total Cost : 6
```